

Define la identidad de tu agente

Este módulo se basa en los siguientes puntos:

- El entorno de agente funcional del módulo 1.
- El código de agente predeterminado creado por `adk create`
- Comprender que modelo + herramientas + organización = agente

Conoce a tu primer agente

En el módulo 1, creaste un agente funcional con `adk create`. Cuando abriste `agent.py`, viste este código:

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='Un agente de asistente útil.',
    instruction='Eres un asistente útil.'
)
```

Verificaste que funciona con `adk web`. Es un éxito. Pero ¿qué significan realmente estos parámetros? ¿Cómo puedes personalizar este agente para tareas específicas? Este módulo sirve para comprender el componente “Modelo” del curso 1, que es el cerebro de razonamiento de tu agente.

Los cuatro parámetros principales

Referencia: [Documentos de ADK - Agente de LLM: Definición de la identidad y el propósito del agente](#)

Cada `LlmAgent` (también importado como `Agent`) se define por cuatro parámetros principales. Exploreemos cada uno.

1. `model` (obligatorio)

Qué es: El modelo de lenguaje grande (LLM) subyacente que impulsa el razonamiento y la toma de decisiones de tu agente.

Ejemplo:

```
model='gemini-2.5-flash'
```

Qué hace:

- Determina la inteligencia y las capacidades de tu agente.
- Afecta el costo por solicitud y la velocidad de respuesta.
- Los diferentes modelos tienen diferentes fortalezas (velocidad o capacidad).

Modelos disponibles:

- **gemini-2.5-flash**: Es rápido, eficiente y adecuado para la mayoría de las tareas.
- **gemini-2.5-pro**: Más capaz y mejor para el razonamiento complejo
- Consulta la [documentación de modelos](#) para ver la lista completa.

Analogía: Al igual que cuando eliges el motor de tu auto, los motores más potentes pueden manejar tareas más complejas, pero pueden ser más costosos.

Importante: Este es un **parámetro obligatorio**. Tu agente no puede funcionar si no se especifica un modelo.

2. **name** (obligatorio)

Qué es: Es un identificador de cadena único para tu agente.

Ejemplo:

```
name= 'root_agent'
```

Qué hace:

- Identifica a tu agente internamente en el ADK.
- Es fundamental en sistemas multiagente en los que los agentes hacen referencia unos a otros.
- Se usa para el registro, la depuración y la delegación de agentes.

Convenciones de nombres:

-  Usa letras minúsculas con guiones bajos: **customer_support_agent**
-  Sé descriptivo: **data_analysis_agent**, **math_tutor_agent**
-  Evita los nombres reservados: **user**
-  No uses mayúsculas mediales: Usa **my_agent** en lugar de **myAgent**

La documentación del ADK brinda la siguiente información:

“Cada agente necesita un identificador de cadena único. Este nombre es fundamental para las operaciones internas, especialmente en los sistemas multiagente, en los que los agentes deben referirse o delegar tareas entre sí”.

Importante: Este es un **parámetro obligatorio**. Todos los agentes deben tener un nombre.

3. **description** (opcional, recomendado para los sistemas multiagente)

Qué es: Un resumen conciso de lo que hace tu agente.

Ejemplo:

```
description='Responde las preguntas de los usuarios sobre la capital de un país determinado.'
```

Qué hace:

- **Otros agentes la usan para determinar si deben enrutar tareas a este agente.**
- Ayuda en sistemas multiagente en los que los agentes se delegan tareas entre sí.
- **No lo usa el agente en sí** para su propio comportamiento

La documentación del ADK brinda la siguiente información:

“Esta descripción la utilizan principalmente otros agentes de LLM para determinar si deben enrutar una tarea a este agente. Escribe algo lo suficientemente específico para diferenciarlo de los demás”.

Cuándo se usa:

-  Crear sistemas multiagente con delegación
-  Agentes a los que llamarán otros agentes
-  Menos importante para aplicaciones de un solo agente

Buenas descripciones:

-  “Atiende las consultas de facturación de los clientes y procesa las actualizaciones de pagos”.
-  “Analiza los datos de ventas y genera informes de rendimiento semanales”.
-  “Ayuda a los estudiantes a aprender álgebra guiándolos a través de los pasos para resolver problemas”.
-  “Agente de facturación” (demasiado básico)
-  “Ayudante” (no es lo suficientemente específico)

4. **instruction** (muy importante, pero opcional)

Qué es: Es el plano de comportamiento que guía cómo actúa y responde tu agente.

Ejemplo:

```
instruction="""Eres un asistente útil que dice la hora actual de las ciudades. Usa la herramienta 'get_current_time' para este propósito."""
```

Qué hace:

- Define la personalidad y el estilo de comunicación del agente.
- Especifica la tarea o el objetivo principal del agente.
- Establece límites y restricciones en el comportamiento.
- Indica cuándo y cómo usar las herramientas (este tema se aborda en el módulo 3).
- Da forma al formato de salida.

La documentación del ADK brinda la siguiente información:

“El parámetro instructions es, sin duda, el más importante para definir el comportamiento de un LlmAgent. Le indica al agente su tarea o meta principal, su personalidad o arquetipo, las restricciones de su comportamiento y cómo y cuándo usar sus herramientas”.

Sugerencias para crear instrucciones eficaces (de la documentación del ADK):

- **Exprésate de forma clara y específica:** Evita la ambigüedad y establece claramente las acciones y los resultados deseados.
- **Usa markdown:** Mejora la legibilidad de instrucciones complejas con encabezados, listas, etcétera.
- **Proporciona ejemplos (instrucción con varios ejemplos):** Para tareas complejas o formatos de salida específicos, incluye ejemplos.
- **Indica el uso de las herramientas:** No te limites a enumerar las herramientas, explica cuándo y por qué el agente debería usarlas.

Los parámetros **description** y **instruction** en comparación: La diferencia clave

Es fundamental comprender la siguiente información:

Parámetro	Público	Objetivo	Ejemplo
description	Otros agentes	“¿Debería enrutar esta tarea aquí?”	“Atiende consultas de facturación”.
instruction	Este agente	“¿Cómo debo comportarme?”	“Eres un experto en facturación que...”

Ejemplo concreto:

```
# Sistema multiagente con 3 agentes
billing_agent = Agent(
    model='gemini-2.5-flash',
    name='billing_agent',
    # OTROS agentes leen esto para determinar si deberían delegar tareas aquí
    description='Se encarga de las consultas sobre facturación de los clientes y el
```

```

procesamiento de los pagos',
    # ESTE agente lee esto para saber cómo comportarse
    instruction="""Eres un especialista en facturación.

    Cuando ayudes a los clientes:
    1. Sé comprensivo y paciente.
    2. Explica los cargos con claridad.
    3. Usa la herramienta billing_lookup para verificar los detalles de la cuenta.
    4. Nunca prometas reembolsos sin la aprobación del gerente.

    Mantén siempre un tono profesional y servicial."""
)

support_agent = Agent(
    model='gemini-2.5-flash',
    name='support_agent',
    description='Se encarga de las preguntas generales de asistencia al cliente,
    instruction="""Eres un agente de asistencia al cliente.

    Si la pregunta es sobre facturación, transfiere la llamada al agente de
    facturación.
    De lo contrario, ayuda al cliente directamente."""
)

```

En este ejemplo:

- `support_agent` lee la **descripción** de `billing_agent` para decidir: “¿Es una pregunta sobre facturación?”
- `billing_agent` lee su propia **instrucción** para saber: “¿Cómo manejo las preguntas de facturación?”

La convención de nombres de `root_agent`

Referencia: [Guía de inicio rápido de Python para el ADK](#)

Es posible que hayas notado que tu variable de agente se llama `root_agent`:

```

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent', # Nombre interno (dentro de Agent)
    # ...
)

```

¿Por qué `root_agent`?

Las herramientas de línea de comandos del ADK buscan una variable de Python llamada `root_agent` como punto de entrada a tu sistema de agentes. Esta es una convención que permite que el ADK descubra y ejecute tu agente.

La documentación del ADK brinda la siguiente información:

“El archivo agent.py contiene una definición de `root_agent`, que es el único elemento obligatorio de un agente de ADK”.

¿Puede el nombre interno ser diferente del nombre de la variable?

Sí. El parámetro `name` dentro de `Agent()` es independiente del nombre de la variable:

```
# Nombre interno del ADK (se usa para los registros y la delegación)
my_specialized_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent', # Lo usa el ADK internamente
    description='Ayuda a los estudiantes con álgebra',
    instruction='Eres un tutor de matemáticas paciente...'
)

# Nombre de la variable que buscan las herramientas del ADK (debe ser root_agent)
root_agent = my_specialized_agent
```

Regla clave: Siempre asigna tu agente principal a una variable llamada `root_agent` para que las herramientas del ADK puedan encontrarlo.

En el módulo 3, aprenderás cómo las herramientas del ADK usan `root_agent` para ejecutar tu agente de diferentes maneras.

Escribe instrucciones eficaces

Referencia: [Documentos del ADK - Guía del agente: Instrucciones](#)

Las instrucciones guían el comportamiento y la personalidad de tu agente. Para este curso, usaremos instrucciones simples para ayudarte a comprender los conceptos básicos.

Enfoque simple para el curso 2:

```
instruction="Eres un tutor de matemáticas paciente. Ayuda a los estudiantes con problemas de álgebra".
```

Esta instrucción básica es suficiente para comenzar. Le indica al agente los siguientes aspectos:

- Su rol (tutor de matemáticas)
- Su rasgo de personalidad (paciente)
- Su tarea (ayudar con álgebra)

¿Quieres aprender patrones de instrucciones profesionales? En el **curso 3: Cómo definir un mejor comportamiento del agente**, aprenderás técnicas avanzadas, como las que se enumeran a continuación:

- Estructuras de instrucciones de varias secciones
- Definiciones de límites y personalidades

- Instrucciones con varios ejemplos
- Patrones de instrucciones listos para la producción

Por ahora, enfoquémonos en comprender cómo funcionan las instrucciones básicas.